

Banking Transaction Analysis:

A bank provides Customer, Account, Transaction, Loan, and Branch datasets.

Analyze transaction patterns, customer groups, loan information, branch performance, and monthly transaction trends. Identify unusual transactions, correlations, outliers, and important financial insights.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

customer = pd.read_csv("Customer.csv")
account = pd.read_csv("Account.csv")
transaction = pd.read_csv("Transaction.csv")
loan = pd.read_csv("Loan.csv")
branch = pd.read_csv("Branch.csv")

print("Customer Dataset")
print(customer.head())

print("\nAccount Dataset")
print(account.head())

print("\nTransaction Dataset")
print(transaction.head())

print("\nLoan Dataset")
print(loan.head())

print("\nBranch Dataset")
print(branch.head())

print("\nMissing Values")
print(transaction.isnull().sum())

print("\nDuplicate Records")
print(transaction.duplicated().sum())

transaction = transaction.drop_duplicates()

# Convert date
transaction["TransactionDate"] = pd.to_datetime(
    transaction["TransactionDate"])
```

```

)

print("\nTransaction Statistics")
print(transaction["Amount"].describe())

print("\nTransaction Type Count")
print(transaction["TransactionType"].value_counts())
transaction["Month"] = transaction["TransactionDate"].dt.to_period("M")
monthly_transactions = transaction.groupby(
    "Month"
)["Amount"].sum()
print("\nMonthly Transaction Amount")
print(monthly_transactions)
plt.figure(figsize=(10, 5))
monthly_transactions.plot(marker="o")
plt.title("Monthly Transaction Trend")
plt.xlabel("Month")
plt.ylabel("Total Transaction Amount")
plt.xticks(rotation=45)
plt.grid()
plt.show()
customer_group = customer.groupby("Gender").agg(
    Customers=("CustomerID", "count"),
    AverageIncome=("Income", "mean")
)
print("\nCustomer Group Analysis")
print(customer_group)
plt.figure(figsize=(7, 5))
sns.barplot(
    x=customer_group.index,
    y=customer_group["Customers"]
)
plt.title("Customers by Gender")

```

```

plt.ylabel("Number of Customers")
plt.show()

account_summary = account.groupby(
    "AccountType"
)["Balance"].agg(["count", "mean", "sum"])
print("\nAccount Analysis")
print(account_summary)
plt.figure(figsize=(8, 5))
account_summary["sum"].plot(kind="bar")
plt.title("Total Balance by Account Type")
plt.xlabel("Account Type")
plt.ylabel("Total Balance")
plt.xticks(rotation=0)
plt.show()
print("\nLoan Status")
print(loan["LoanStatus"].value_counts())
loan_summary = loan.groupby(
    "LoanType"
)["LoanAmount"].agg(["count", "mean", "sum"])
print("\nLoan Analysis")
print(loan_summary)
plt.figure(figsize=(8, 5))
loan_summary["sum"].plot(kind="bar")
plt.title("Total Loan Amount by Loan Type")
plt.ylabel("Loan Amount")
plt.show()

branch_transaction = transaction.merge(
    account,
    on="AccountID",
    how="left"
)
branch_transaction = branch_transaction.merge(

```

```

customer[["CustomerID", "City"],
on="CustomerID",
how="left"
)
branch_performance = branch_transaction.groupby(
    "City"
)["Amount"].agg(["count", "sum", "mean"])
print("\nBranch/City Performance")
print(branch_performance)
plt.figure(figsize=(10, 5))
branch_performance["sum"].sort_values(
    ascending=False
).plot(kind="bar")
plt.title("Branch Transaction Performance")
plt.xlabel("City")
plt.ylabel("Total Transaction Amount")
plt.xticks(rotation=45)
plt.show()
Q1 = transaction["Amount"].quantile(0.25)
Q3 = transaction["Amount"].quantile(0.75)
IQR = Q3 - Q1
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR
outliers = transaction[
    (transaction["Amount"] < lower_limit) |
    (transaction["Amount"] > upper_limit)
]
print("\nUnusual Transactions")
print(outliers)
plt.figure(figsize=(10, 5))
sns.boxplot(x=transaction["Amount"])

```

```

plt.title("Transaction Amount Outliers")
plt.xlabel("Transaction Amount")
plt.show()
numeric_data = customer[
    ["Age", "Income"]
].copy()
numeric_data["CustomerID"] = customer["CustomerID"]
merged_data = numeric_data.merge(
    loan[["CustomerID", "LoanAmount"]],
    on="CustomerID",
    how="inner"
)
print("\nCorrelation Matrix")
print(merged_data.corr(numeric_only=True))
plt.figure(figsize=(7, 5))
sns.heatmap(
    merged_data.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm"
)
plt.title("Banking Data Correlation")
plt.show()
print("\n===== BANKING INSIGHTS =====")
print("Total Transactions:",
      len(transaction))
print("Total Transaction Value:",
      transaction["Amount"].sum())
print("Average Transaction:",
      transaction["Amount"].mean())
print("Largest Transaction:",
      transaction["Amount"].max())
print("Number of Unusual Transactions:",

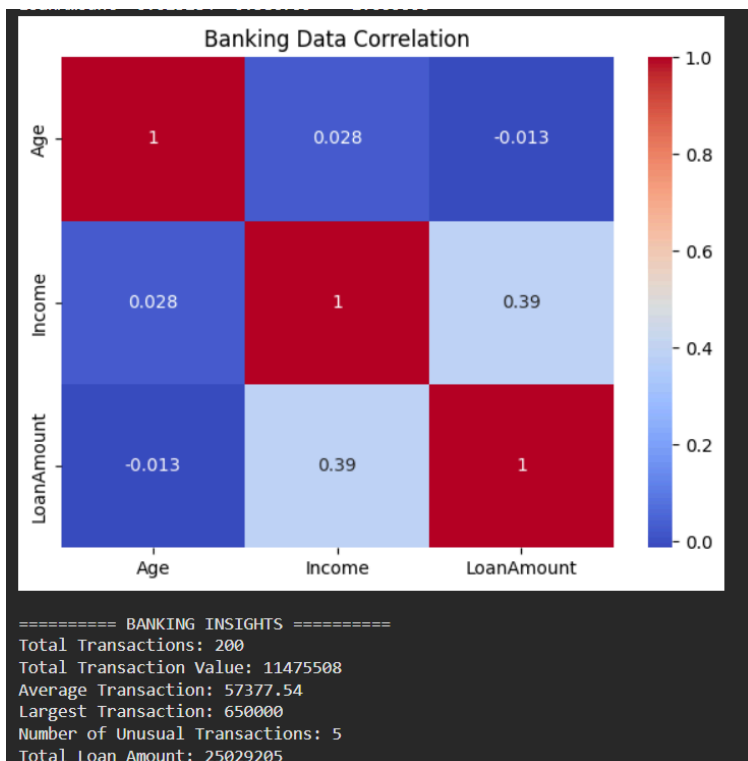
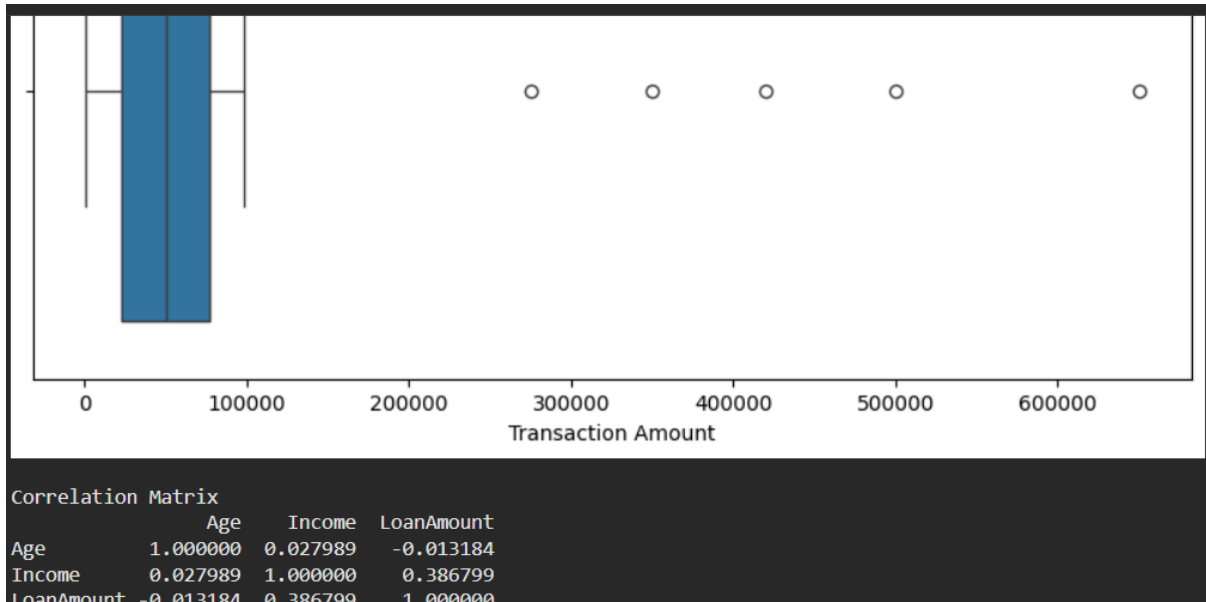
```

```
len(outliers))

print("Total Loan Amount:",

      loan["LoanAmount"].sum())
```

OUTPUT:



Social Media Engagement Analysis:

A social media platform provides User, Post, Hashtag, Like, Comment, and Share datasets. Analyze user engagement, popular content, hashtags, posting times, and engagement trends. Identify correlations and outliers and create suitable visualizations or an interactive dashboard.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

user = pd.read_csv("User.csv")
post = pd.read_csv("Post.csv")
hashtag = pd.read_csv("Hashtag.csv")
likes = pd.read_csv("Like.csv")
comment = pd.read_csv("Comment.csv")
share = pd.read_csv("Share.csv")

print("User Dataset")
print(user.head())

print("\nPost Dataset")
print(post.head())

print("\nHashtag Dataset")
print(hashtag.head())

print("\nLike Dataset")
print(likes.head())
print("\nComment Dataset")
print(comment.head())
print("\nShare Dataset")
```

```
print(share.head())
print("\nMissing Values")
print("Users:")
print(user.isnull().sum())
print("Posts:")
print(post.isnull().sum())
print("Likes:")
print(likes.isnull().sum())
print("Comments:")
print(comment.isnull().sum())
print("Shares:")
print(share.isnull().sum())
user = user.drop_duplicates()
post = post.drop_duplicates()
hashtag = hashtag.drop_duplicates()
likes = likes.drop_duplicates()
comment = comment.drop_duplicates()
share = share.drop_duplicates()

like_count = likes.groupby(
    "PostID"
).size().reset_index(name="Likes")
comment_count = comment.groupby(
    "PostID"
).size().reset_index(name="Comments")
share_count = share.groupby(
    "PostID"
).size().reset_index(name="Shares")
engagement = post.copy()
engagement = engagement.merge(
    like_count,
    on="PostID",
```



```

        how="left"
    )
    engagement = engagement.merge(
        comment_count,
        on="PostID",
        how="left"
    )
    engagement = engagement.merge(
        share_count,
        on="PostID",
        how="left"
    )
    engagement[
        ["Likes", "Comments", "Shares"]
    ] = engagement[
        ["Likes", "Comments", "Shares"]
    ].fillna(0)
    engagement["TotalEngagement"] = (
        engagement["Likes"] +
        engagement["Comments"] +
        engagement["Shares"]
    )
    print("\nPost Engagement")
    print(engagement.head())
    popular_posts = engagement.sort_values(
        "TotalEngagement",
        ascending=False
    )
    print("\nMost Popular Posts")
    print(
        popular_posts[
            ["PostID", "ContentType",

```

```

        "Likes", "Comments",
        "Shares", "TotalEngagement"]
    ].head(10)
)
content_analysis = engagement.groupby(
    "ContentType"
)[
    ["Likes", "Comments", "Shares",
    "TotalEngagement"]
].mean()
print("\nContent Type Engagement")
print(content_analysis)
plt.figure(figsize=(8, 5))
content_analysis["TotalEngagement"].sort_values(
    ascending=False
).plot(kind="bar")
plt.title("Average Engagement by Content Type")
plt.xlabel("Content Type")
plt.ylabel("Average Engagement")
plt.xticks(rotation=0)
plt.show()
hashtag_count = hashtag.groupby(
    "Hashtag"
).size().sort_values(
    ascending=False
)
print("\nTop Hashtags")
print(hashtag_count.head(10))

plt.figure(figsize=(10, 5))

hashtag_count.head(10).sort_values().plot(

```

```

        kind="barh"
    )

plt.title("Top 10 Hashtags")
plt.xlabel("Number of Posts")
plt.ylabel("Hashtag")
plt.show()

hashtag_engagement = hashtag.merge(
    engagement,
    on="PostID",
    how="inner"
)

hashtag_performance = hashtag_engagement.groupby(
    "Hashtag"
)[["TotalEngagement"].mean().sort_values(
    ascending=False
)]

print("\nBest Performing Hashtags")
print(hashtag_performance.head(10))

post["PostDate"] = pd.to_datetime(
    post["PostDate"]
)

post["Hour"] = pd.to_datetime(
    post["PostTime"],
    format="%H:%M:%S",
    errors="coerce"
).dt.hour

```

```
time_analysis = engagement.copy()

time_analysis["Hour"] = post["Hour"]

hourly_engagement = time_analysis.groupby(
    "Hour"
)["TotalEngagement"].mean()

print("\nEngagement by Posting Hour")
print(hourly_engagement)

plt.figure(figsize=(10, 5))

hourly_engagement.plot(
    marker="o"
)

plt.title("Average Engagement by Posting Time")
plt.xlabel("Hour of Day")
plt.ylabel("Average Engagement")
plt.grid()
plt.show()

engagement["PostDate"] = pd.to_datetime(
    engagement["PostDate"]
)

daily_engagement = engagement.groupby(
    "PostDate"
)["TotalEngagement"].sum()

plt.figure(figsize=(12, 5))
```

```
daily_engagement.plot()
```

```
plt.title("Daily Social Media Engagement Trend")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Total Engagement")
```

```
plt.grid()
```

```
plt.show()
```

```
user_engagement = engagement.merge(  
    user,  
    on="UserID",  
    how="left"  
)
```

```
user_summary = user_engagement.groupby(  
    "Username"  
)[  
    ["Likes", "Comments",  
     "Shares", "TotalEngagement"]  
].sum().sort_values(  
    "TotalEngagement",  
    ascending=False  
)
```

```
print("\nTop Engaging Users")
```

```
print(user_summary.head(10))
```

```
correlation_data = engagement[  
    ["Likes", "Comments",  
     "Shares", "TotalEngagement"]  
]
```

```
print("\nCorrelation Matrix")
```

```
print(correlation_data.corr())
```

```

plt.figure(figsize=(7, 5))

sns.heatmap(
    correlation_data.corr(),
    annot=True,
    cmap="coolwarm"
)

plt.title("Social Media Engagement Correlation")

plt.show()

Q1 = engagement[
    "TotalEngagement"
].quantile(0.25)

Q3 = engagement[
    "TotalEngagement"
].quantile(0.75)

IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR

upper = Q3 + 1.5 * IQR

outliers = engagement[
    (engagement["TotalEngagement"] < lower) |
    (engagement["TotalEngagement"] > upper)
]

print("\nEngagement Outliers")

print(outliers[
    ["PostID", "Likes",
     "Comments", "Shares",
     "TotalEngagement"]
])

plt.figure(figsize=(10, 5))

sns.boxplot(
    x=engagement["TotalEngagement"]
)

```

```

plt.title("Social Media Engagement Outliers")
plt.xlabel("Total Engagement")
plt.show()
engagement[
    ["Likes", "Comments", "Shares"]
].sum().plot(
    kind="bar"
)
plt.title("Total Likes, Comments and Shares")
plt.xlabel("Engagement Type")
plt.ylabel("Count")
plt.xticks(rotation=0)
plt.show()
print("\n===== SOCIAL MEDIA INSIGHTS =====")
print(
    "Total Users:",
    user["UserID"].nunique()
)
print(
    "Total Posts:",
    post["PostID"].nunique()
)
print(
    "Total Likes:",
    len(likes)
)
print(
    "Total Comments:",
    len(comment)
)
print(
    "Total Shares:",

```

```
    len(share)
)
print(
    "Average Post Engagement:",
    engagement["TotalEngagement"].mean()
)
print(
    "Highest Engagement:",
    engagement["TotalEngagement"].max()
)
print(
    "Number of Engagement Outliers:",
    len(outliers)
)
print(
    "Best Posting Hour:",
    hourly_engagement.idxmax()
)
print(
    "Best Content Type:",
    content_analysis[
        "TotalEngagement"
    ].idxmax()
)
print(
    "Most Used Hashtag:",
    hashtag_count.idxmax()
)
```


OUTPUT:

User Dataset

UserID	Username	Age	Gender	Location	
0	U001	user_1	40	Female	Hyderabad
1	U002	user_2	18	Male	Hyderabad
2	U003	user_3	32	Male	Chennai
3	U004	user_4	38	Female	Mumbai
4	U005	user_5	26	Female	Bengaluru

Post Dataset

PostID	UserID	PostDate	PostTime	ContentType	
0	P0001	U040	2026-06-21	16:45:00	Reel
1	P0002	U002	2026-06-28	02:45:00	Image
2	P0003	U045	2026-04-12	12:00:00	Reel
3	P0004	U025	2026-04-08	13:45:00	Reel
4	P0005	U039	2026-01-12	23:00:00	Image

Hashtag Dataset

HashtagID	PostID	Hashtag	
0	H0001	P0001	#Food
1	H0002	P0001	#Travel
2	H0003	P0002	#AI
3	H0004	P0002	#Food
4	H0005	P0003	#Food

Like Dataset

LikeID	PostID	UserID	
0	LI0001	P0001	U023
1	LI0002	P0001	U008
2	LI0003	P0001	U003
3	LI0004	P0001	U014
4	LI0005	P0001	U002

Comment Dataset

CommentID	PostID	UserID	CommentText	
0	C00001	P0001	U043	Comment on P0001
1	C00002	P0001	U050	Comment on P0001
2	C00003	P0001	U023	Comment on P0001
3	C00004	P0001	U037	Comment on P0001
4	C00005	P0001	U021	Comment on P0001

Share Dataset

ShareID	PostID	UserID	
0	SH0001	P0001	U003
1	SH0002	P0001	U041
2	SH0003	P0001	U012
3	SH0004	P0002	U026
4	SH0005	P0002	U048

Missing Values

Users:

UserID

0

Username

0

Age

0

Gender

0

Location

0

dtype: int64

Posts:

PostID

0

UserID

0

PostDate

0

PostTime

0

ContentType

0

dtype: int64

Likes:

LikeID

0

PostID

0

UserID

0

dtype: int64

Comments:

CommentID

0

PostID

0

UserID

0

CommentText

0

dtype: int64

Shares:

ShareID

0

PostID

0

UserID

0

dtype: int64

Post Engagement

PostID	UserID	PostDate	PostTime	ContentType	Likes	Comments	Shares	\
0	P0001	U040	2026-06-21	16:45:00	Reel	38	8	3
1	P0002	U002	2026-06-28	02:45:00	Image	50	11	4
2	P0003	U045	2026-04-12	12:00:00	Reel	50	10	6
3	P0004	U025	2026-04-08	13:45:00	Reel	50	6	4
4	P0005	U039	2026-01-12	23:00:00	Image	30	26	5

TotalEngagement

0

49

1

65

2

66

3

60

4

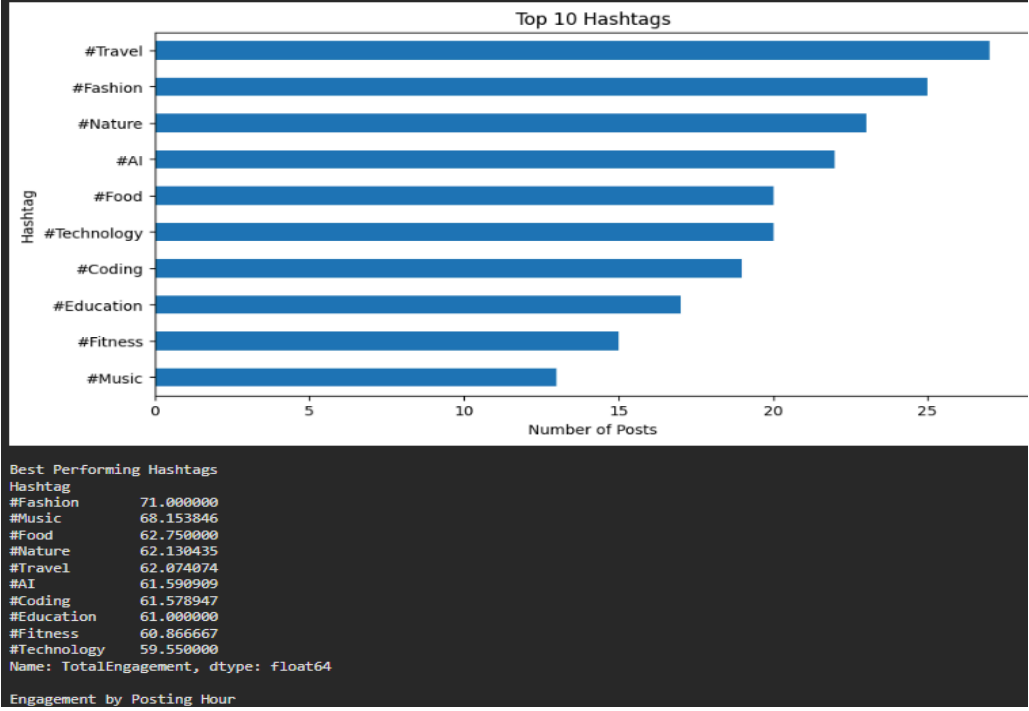
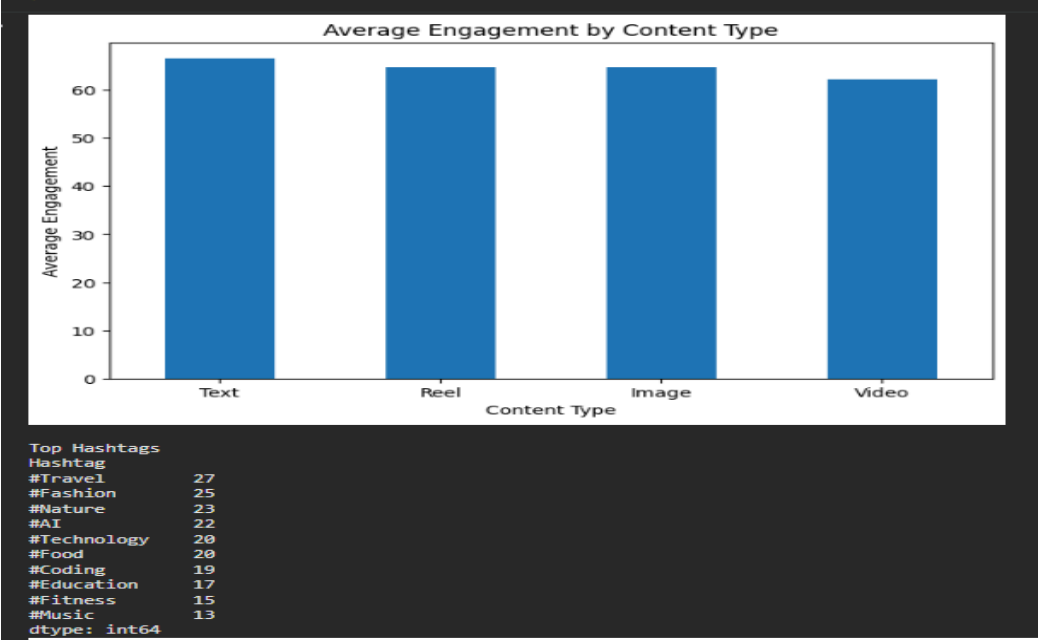
61

Most Popular Posts

PostID	ContentType	Likes	Comments	Shares	TotalEngagement	
10	P0011	Text	50	29	18	97
40	P0041	Reel	50	30	16	96
52	P0053	Video	50	24	19	93
89	P0090	Reel	50	23	20	93
79	P0080	Video	50	28	14	92
50	P0060	Reel	50	22	19	91
91	P0092	Video	50	21	20	91
69	P0070	Image	50	24	15	89
37	P0038	Reel	44	26	19	89
24	P0025	Video	50	21	17	88

Content Type Engagement

	Likes	Comments	Shares	TotalEngagement
ContentType				
Image	38.555556	14.185185	12.000000	64.740741
Reel	40.351351	14.728730	9.675676	64.756757
Text	42.333333	17.222222	7.000000	66.555556
Video	36.025926	15.962963	9.296296	62.185185



Name: TotalEngagement, dtype: float64

